

RNN's

Parte 01: Recorrência Neural

Marcelo Finger
Alan Barzilay

Conteúdos

1. Recorrência Neural
2. Treinamento Recorrente
3. Modelos Sequência pra Sequência
4. Problemas que Advem da Recorrência
5. Soluções para Redes Profundas

NLPortuguês

Recorrência Neural

NLPortuguês

Ordem importa, contexto importa

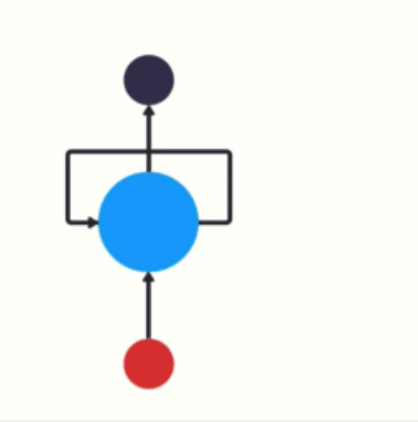
Ao trabalharmos com redes convencionais, não fornecemos nenhuma informação sobre ordem ou sequência dos tokens.

Mas sabemos que o contexto em que as palavras ocorrem em uma frase influencia no seu significado.



Redes Recorrentes

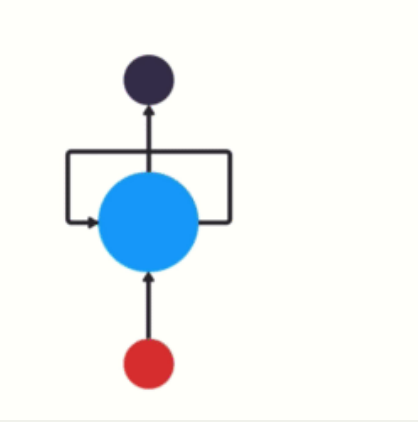
Ao invés de utilizarmos camadas sequenciais que recebem uma frase inteira de uma vez podemos criar redes que recebem uma palavra por vez e se “realimentam” a cada nova palavra.



NLPortuguês

Redes Recorrentes

Deixamos de necessitar *padding* e conseguimos fornecer informação posicional sobre cada palavra através da própria arquitetura da rede.



NLPortuguês

Redes *Feed Forward*

- Cada iteração é independente.

Redes Recorrentes

- Cada iteração depende da anterior.

NLPortuguês

Redes *Feed Forward*

- Cada iteração é independente.
- Facilmente paralelizável.

Redes Recorrentes

- Cada iteração depende da anterior.
- Sequencial por natureza, menor oportunidade de paralelização.

NLPortuguês

Redes *Feed Forward*

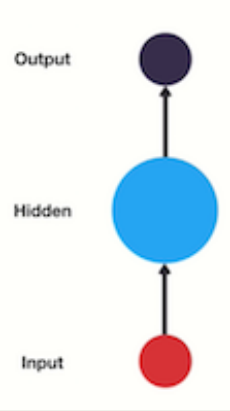
- Cada iteração é independente.
- Facilmente paralelizável.
- Alimentamos toda uma sequência de tokens de uma vez.

Redes Recorrentes

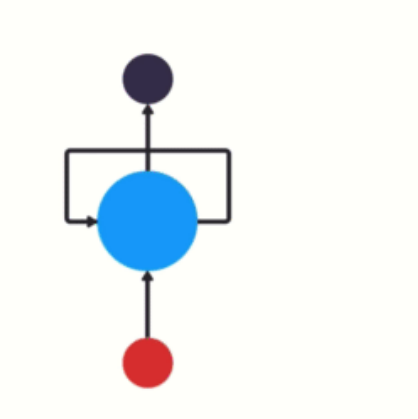
- Cada iteração depende da anterior.
- Sequencial por natureza, menor oportunidade de paralelização.
- Alimentamos um token de cada vez de maneira sequencial

NLPortuguês

Redes *Feed Forward*



Redes Recorrentes



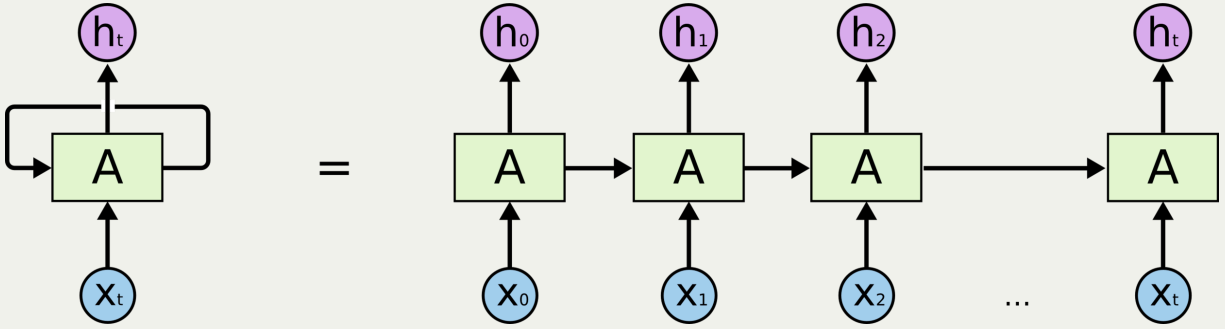
NLPortuguês

Representação “desenrolada”



NLPortuguês

Representação “desenrolada”

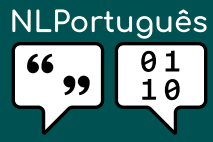


NLPortuguês









RNN's

Parte 02: Treinamento Recorrente

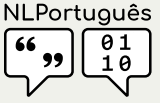
Marcelo Finger
Alan Barzilay

Conteúdos

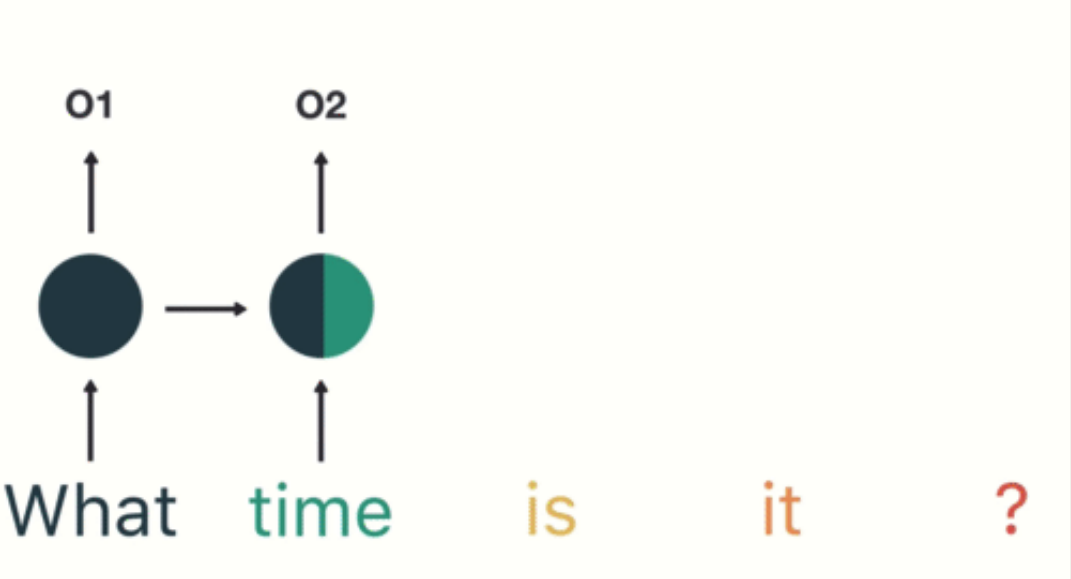
1. Recorrência Neural
2. Treinamento Recorrente
3. Modelos Sequência pra Sequência
4. Problemas que Advem da Recorrência
5. Soluções para Redes Profundas

NLPortuguês

Treinamento Recorrente

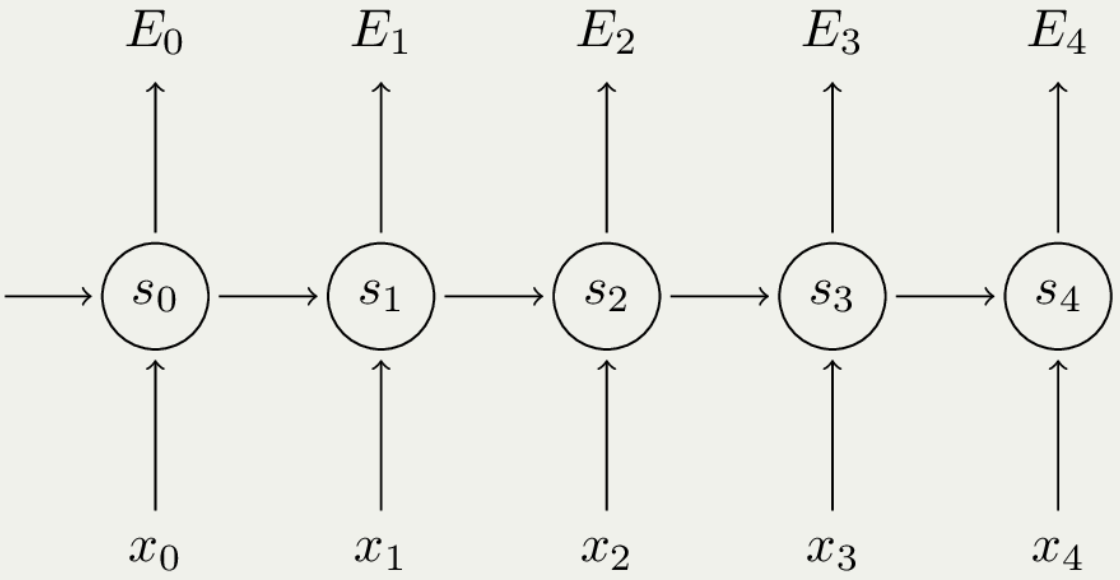


Representação “desenrolada”



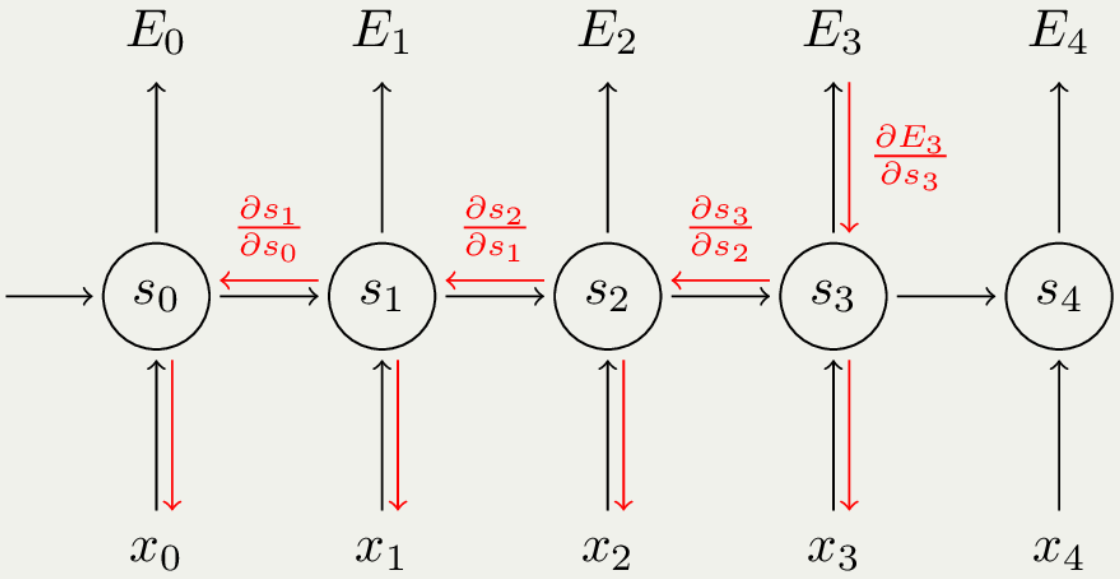
NLPortuguês

Back Propagation Through Time (BPTT)



NLPortuguês

Back Propagation Through Time (BPTT)



NLPortuguês

Recorrência: Modelo Formal

Estado (oculto) seguinte como função da entrada atual e do estado anterior

$$h_t = f(h_{t-1}, x_t)$$

Previsão a ser maximizada: probabilidade da próxima palavra

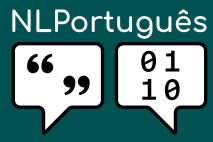
$$p(x_{t,i} = 1 | x_{t-1}, ..., x_1) = \frac{e^{w_i h_t}}{\sum_{j=1}^{K} e^{w_j h_t}}$$

NLPortuguês









RNN's

Parte 03: Modelos Sequência para Sequência

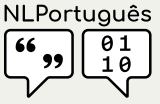
Marcelo Finger
Alan Barzilay

Conteúdos

1. Recorrência Neural
2. Treinamento Recorrente
3. Modelos Sequência pra Sequência
4. Problemas que Advem da Recorrência
5. Soluções para Redes Profundas

NLPortuguês

Modelo Sequence-to-Sequence



Seq2Seq

Diversas tarefas de PLN lidam com sequências:

- Tradução (*Machine Translation*)
- Responder perguntas (*Q&A*)
- Reconhecimento de fala (*Speech Recognition*): tradução de áudio para texto

NLPortuguês

Seq2Seq

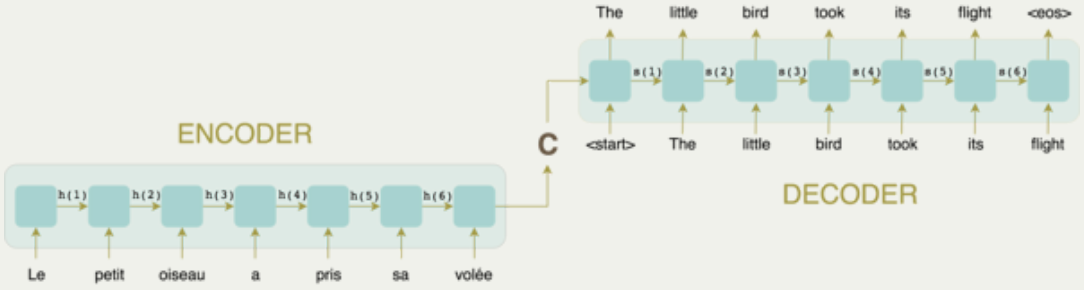
RNN's são capazes não somente de processar sequências como também de gerar sequências.

Uma arquitetura popular para tarefas Seq2Seq é a arquitetura Encoder Decoder.

NLPortuguês

Encoder Decoder

Redes Encoder Decoder possuem 2 partes:

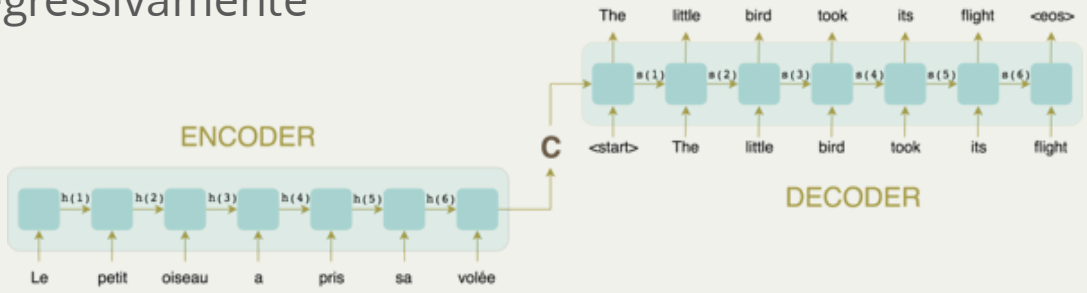


NLPortuguês

Encoder Decoder

Redes Encoder Decoder possuem 2 partes:

- Uma RNN chamada **encoder** que recebe uma sequência de tamanho qualquer e gera uma **codificação**: um vetor **C** de tamanho fixo.
- Uma RNN chamada **decoder** que recebe o vetor **C** e gera uma sequência de tamanho qualquer, auto-regressivamente

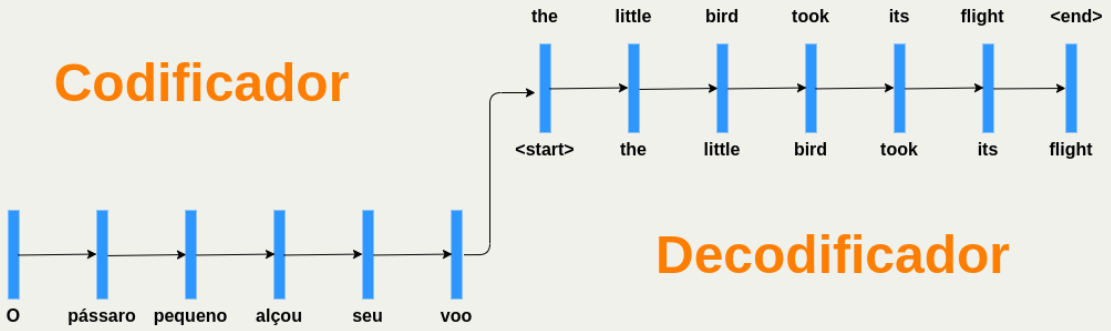


NLPortuguês

Encoder Decoder

Redes Encoder Decoder possuem 2 partes:

- Uma RNN chamada **encoder** que recebe uma sequência de tamanho qualquer e gera uma **codificação**: um vetor **C** de tamanho fixo.

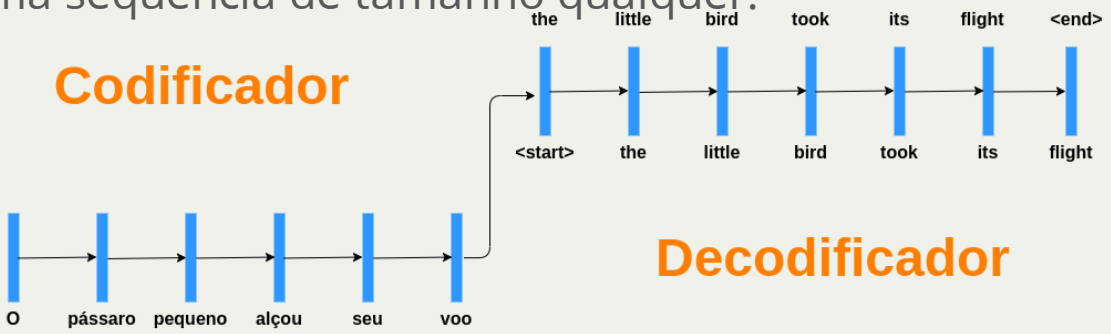


NLPortuguês

Encoder Decoder

Redes Encoder Decoder possuem 2 partes:

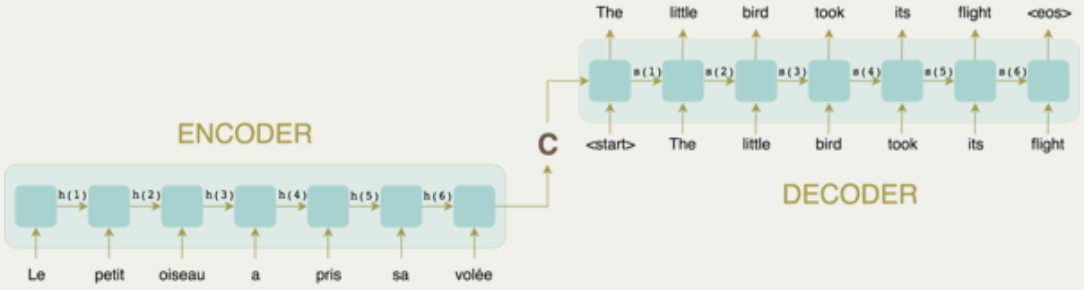
- Uma RNN chamada **encoder** que recebe uma sequência de tamanho qualquer e gera uma **codificação**: um vetor **C** de tamanho fixo.
- Uma RNN chamada **decoder** que recebe o vetor **C** e gera uma sequência de tamanho qualquer.



NLPortuguês

Encoder Decoder

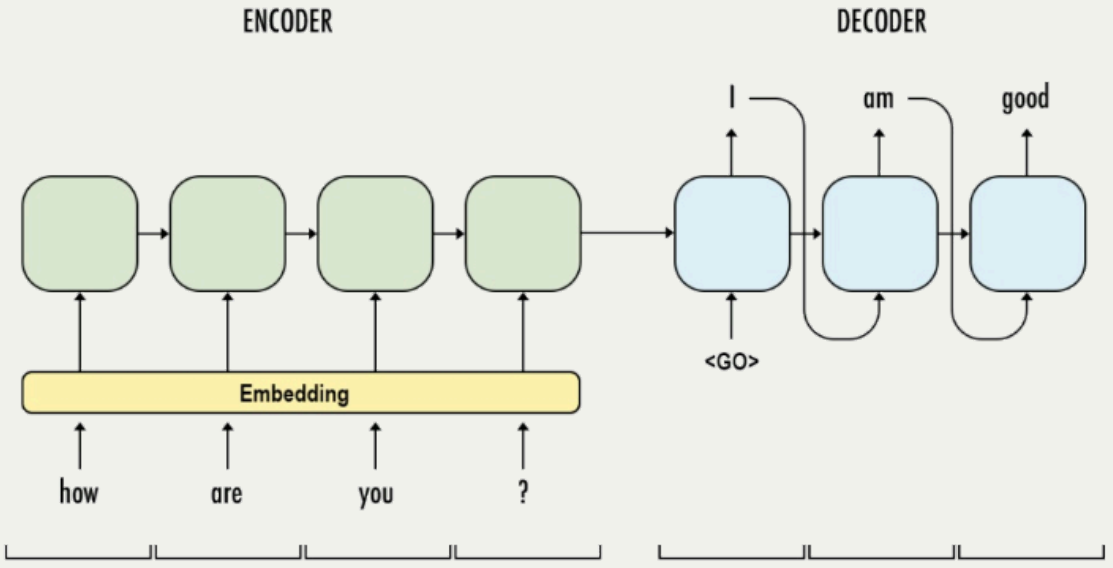
O decoder recebe o último token gerado até prever um token especial **<eos>** que representa o fim de uma frase.



NLPortuguês

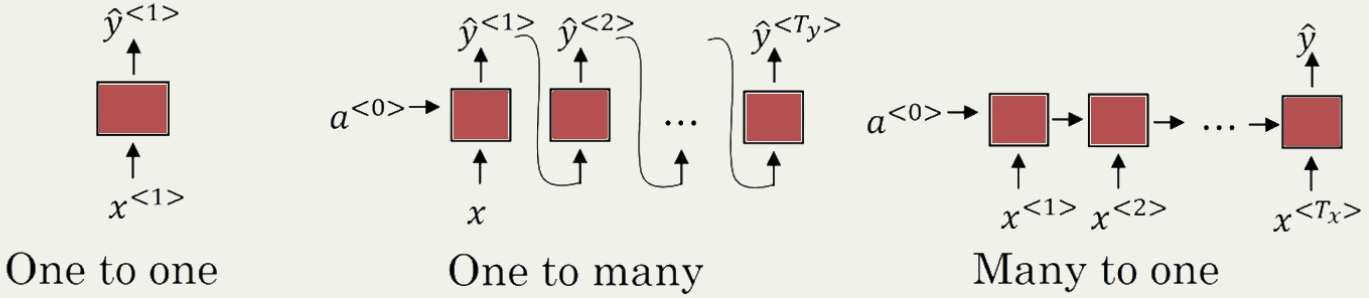
Auto-Alimentação do Decoder

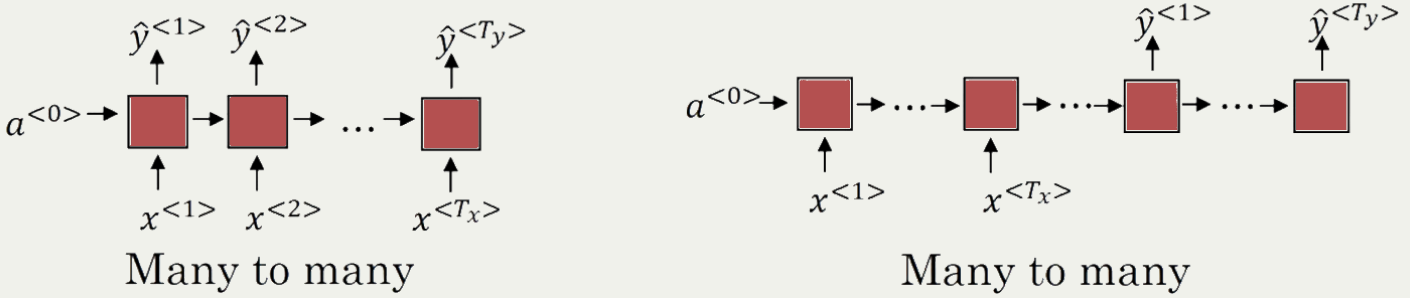
Possibilita a geração de um modelo neural de linguagem



NLPortuguês

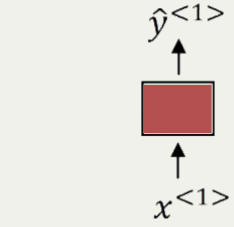
Multiplicidade de Inputs e Outputs



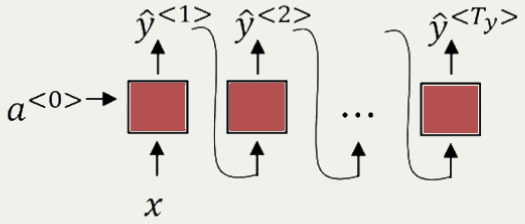


NLPortuguês

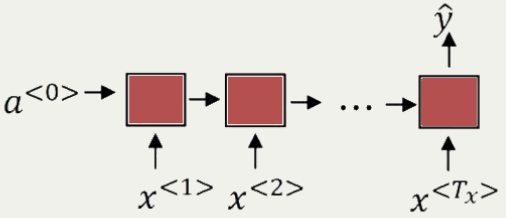
Multiplicidade de Inputs e Outputs



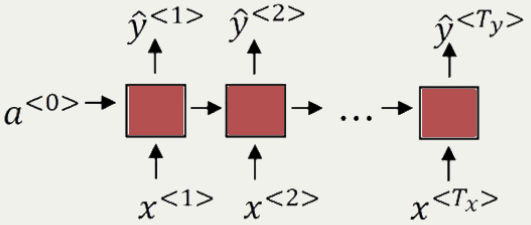
One to one



One to many

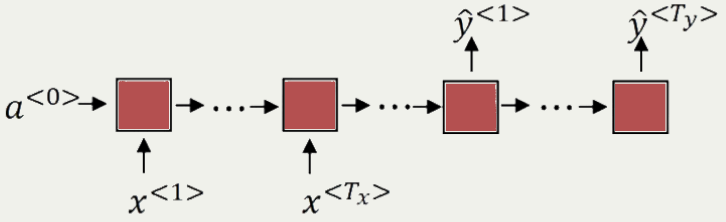


Many to one



Many to many

$T_x = T_y$



Many to many

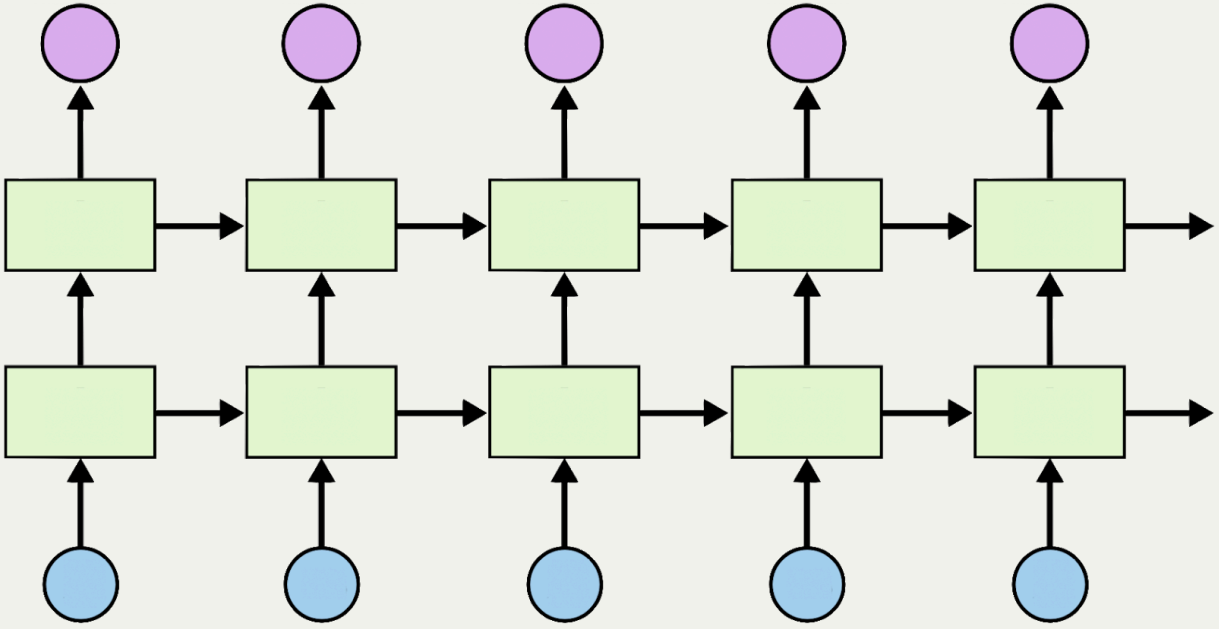
$T_x \neq T_y$ (Encoder-Decoder)

NLPortuguês



NLPortuguês

Ainda podemos adicionar camadas extras

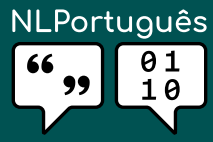


NLPortuguês









RNN's

Parte 04: Problemas Advindos da Recorrência

Marcelo Finger
Alan Barzilay

Conteúdos

1. Recorrência Neural
2. Treinamento Recorrente
3. Modelos Sequência pra Sequência
4. Problemas que Advem da Recorrência
5. Soluções para Redes Profundas

NLPortuguês

Problemas de redes profundas

NLPortuguês

Vanishing & Exploding Gradient

$$\frac{dW}{du} = \frac{dW}{dx}\frac{dx}{dy}\frac{dy}{dz}\frac{dz}{du}$$

O gradiente de uma camada é calculado pela regra da cadeia e portanto é um produto do gradiente de todas as camadas anteriores.

NLPortuguês

Vanishing & Exploding Gradient

$$\frac{dW}{du} = \frac{dW}{dx}\frac{dx}{dy}\frac{dy}{dz}\frac{dz}{du}$$

O que acontece com uma camada se alguma das camadas anteriores possuir um gradiente muito pequeno? E se ele for muito grande?

NLPortuguês

Exploding Gradient

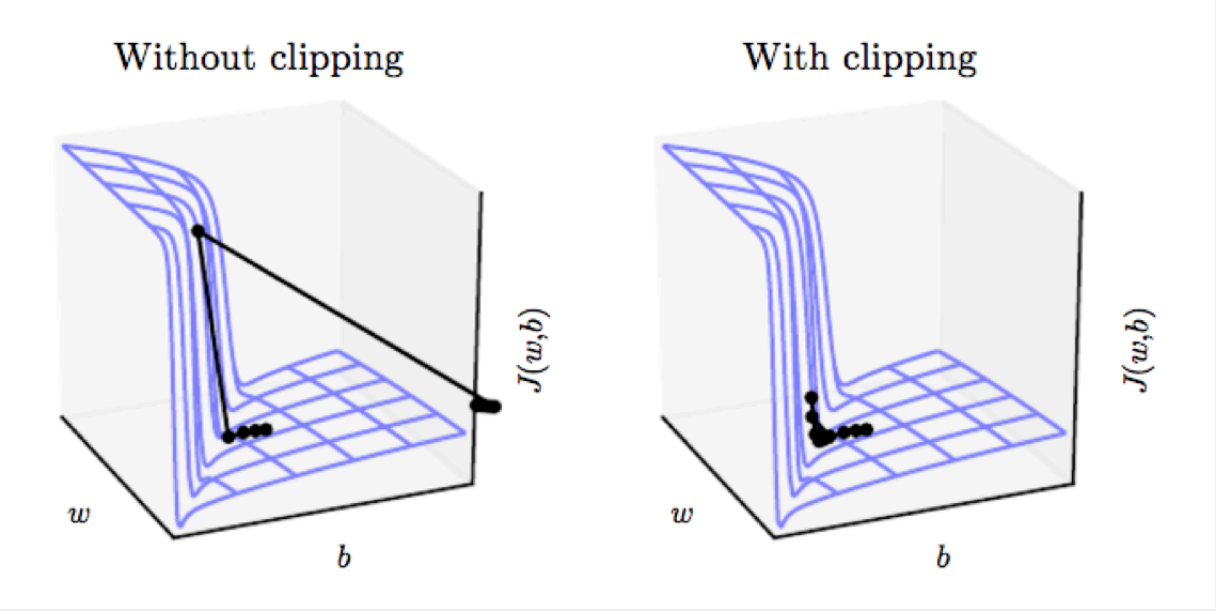
Com um gradiente muito elevado é possível que ao realizarmos uma etapa de *backpropagation* tomemos um passo muito grande que gere um update ruim (aumente a *Loss*).

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

NLPortuguês

Gradient Clipping

Se o passo for acima de um limite definido, nós diminuímos a magnitude do passo e mantemos a direção.



NLPortuguês

Vanishing Gradient

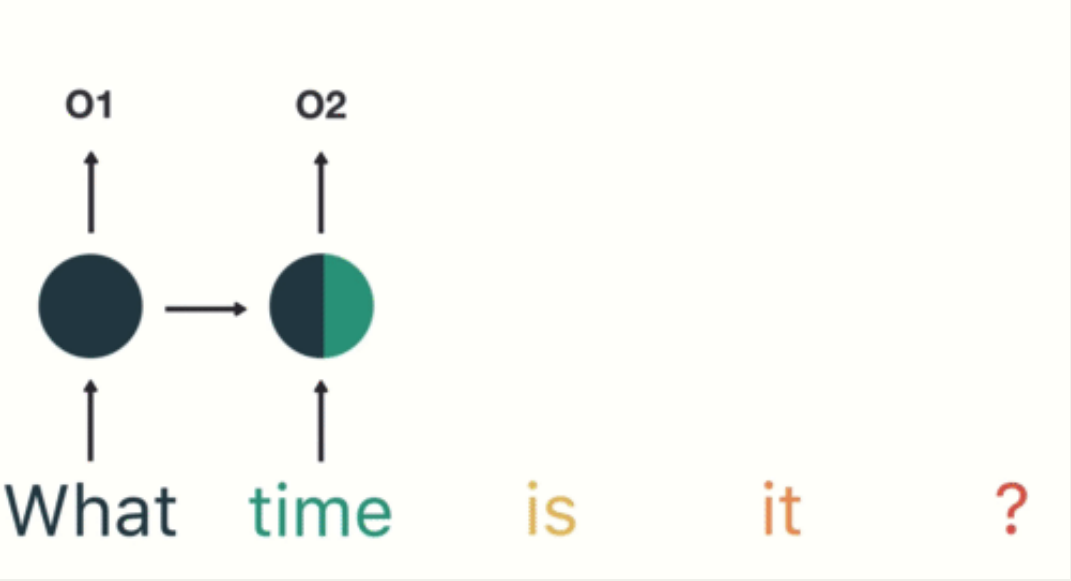
O gradiente pode ser visto como uma medida da influência do passado no futuro.

Com um gradiente baixo a influência passa a ser baixa. Impossível capturar relações distantes.



NLPortuguês

Vanishing Gradient



NLPortuguês

Isso é um problema apenas de redes recorrentes?

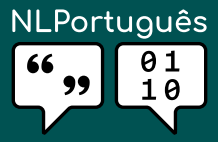
Não! Toda rede profunda sofre destas questões, mas redes recorrentes são especialmente instáveis devido a utilização da mesma matriz de pesos.

NLPortuguês









RNN's

Parte 05: Soluções para Redes Profundas

Marcelo Finger
Alan Barzilay

Conteúdos

1. Recorrência Neural
2. Treinamento Recorrente
3. Modelos Sequência pra Sequência
4. Problemas que Advem da Recorrência
5. Soluções para Redes Profundas

NLPortuguês

Soluções para Redes Profundas

NLPortuguês

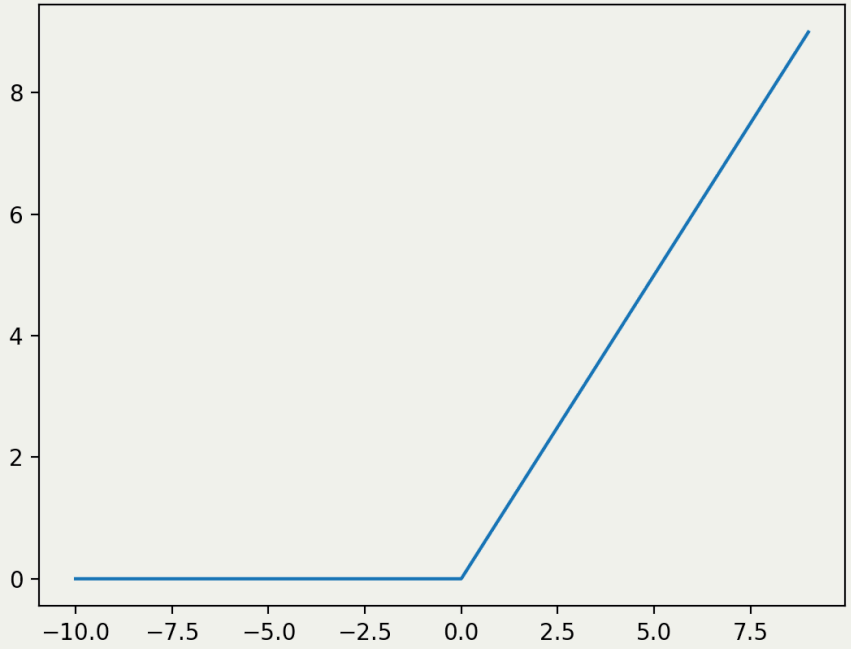
Como combater Vanishing Gradient?

- ReLu

NLPortuguês

ReLu

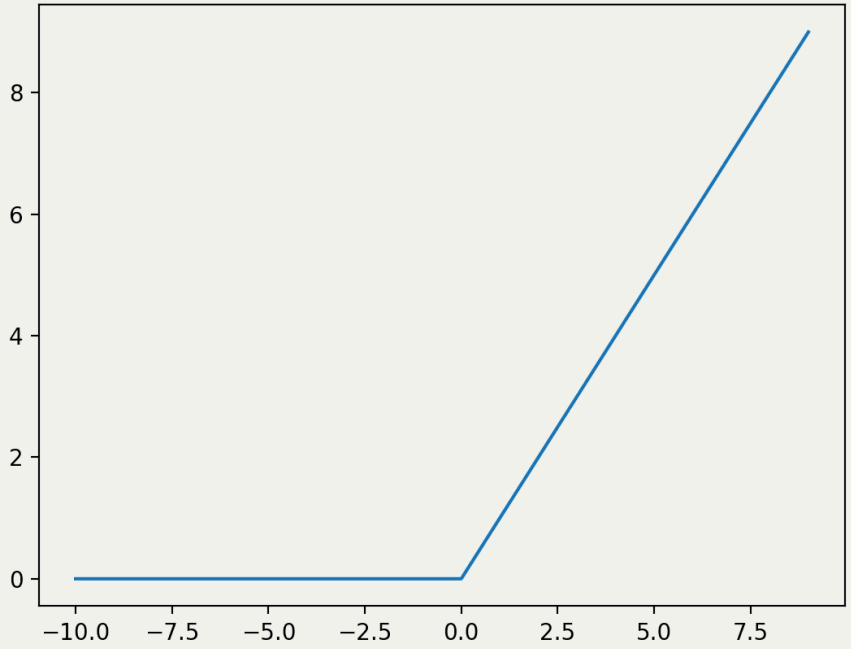
$$ReLu(x) = max(0, x)$$



NLPortuguês

ReLu

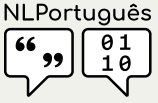
Gradiente não fica saturado como $tanh(x)$



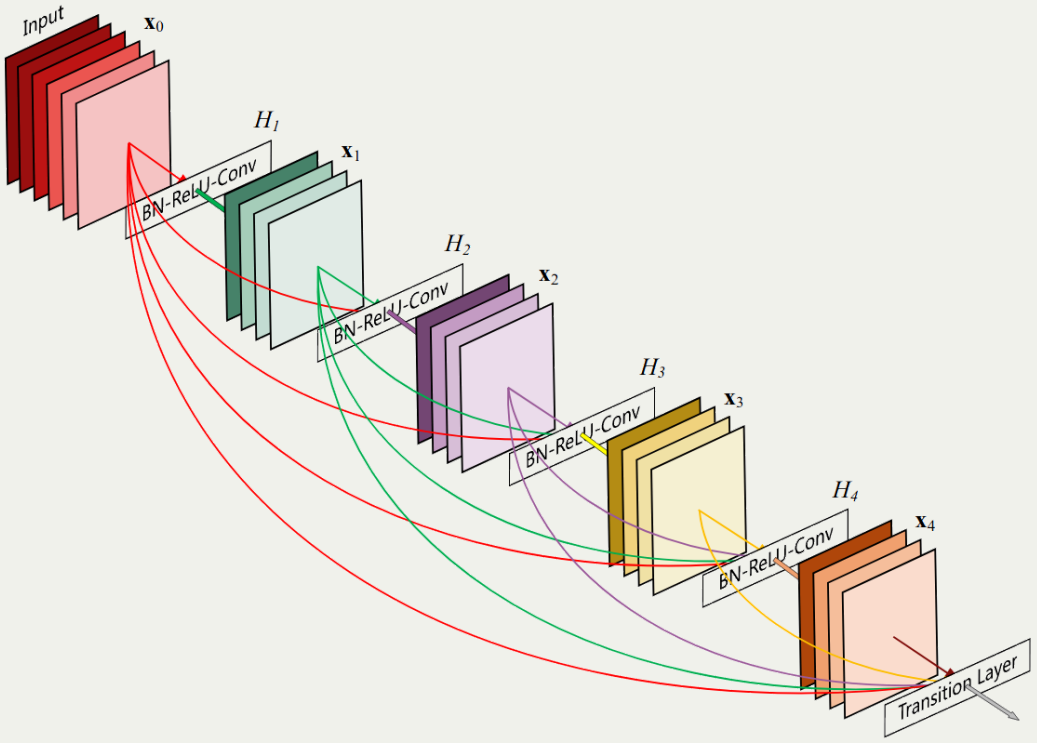
NLPortuguês

Como combater Vanishing Gradient?

- ReLu
- Conexões Densas

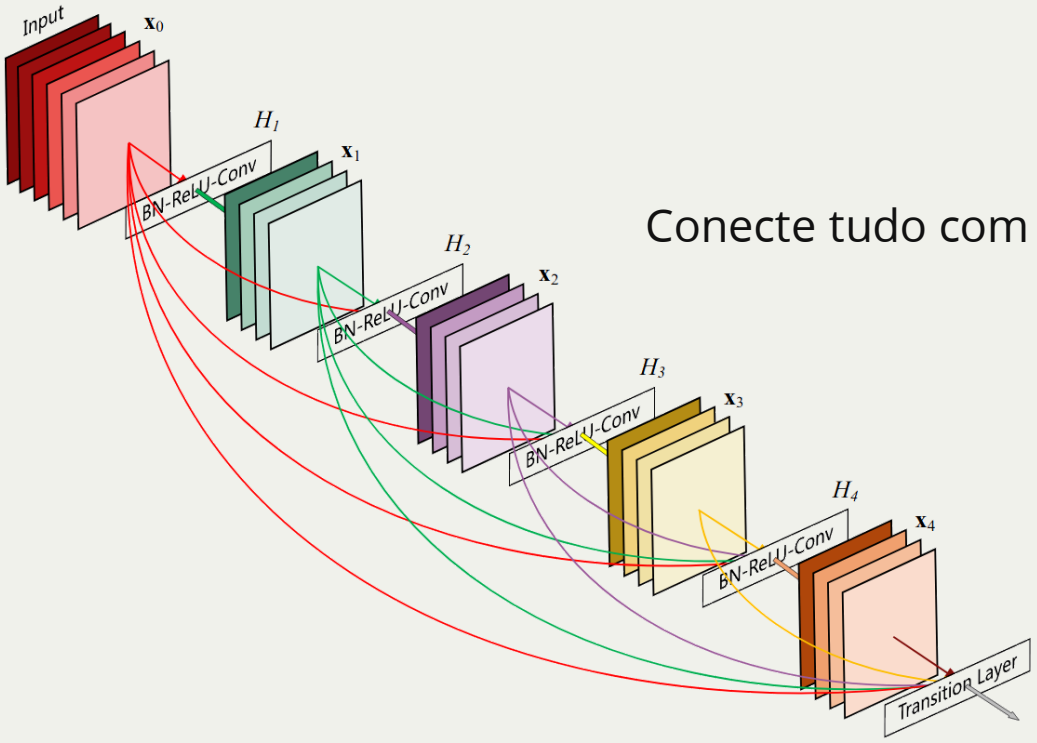


Conexões Densas



NLPortuguês

Conexões Densas



Conecte tudo com tudo

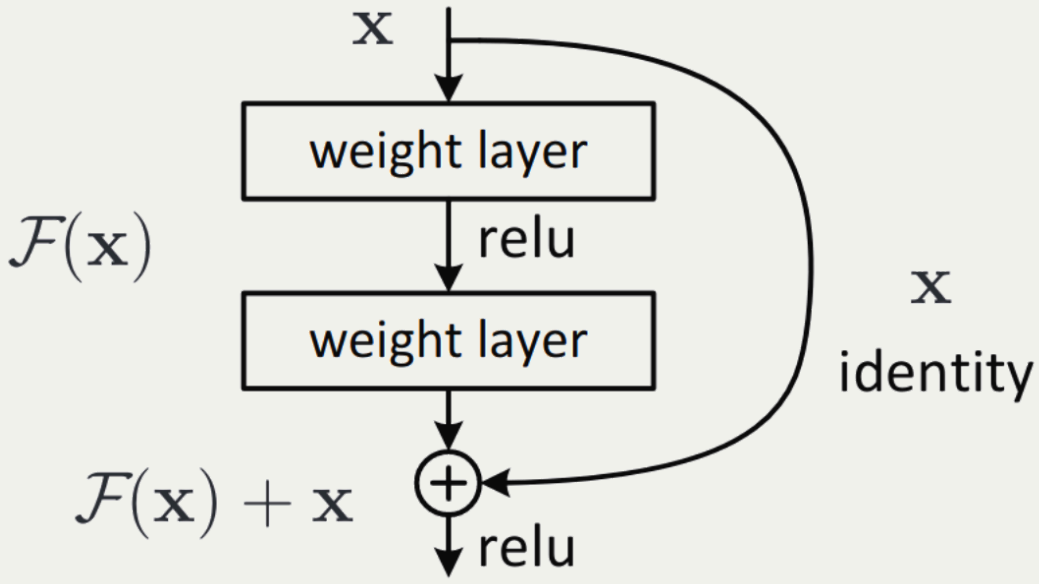
NLPortuguês

Como combater Vanishing Gradient?

- ReLu
- Conexões Densas
- Conexões Residuais
  (Residual Connections)

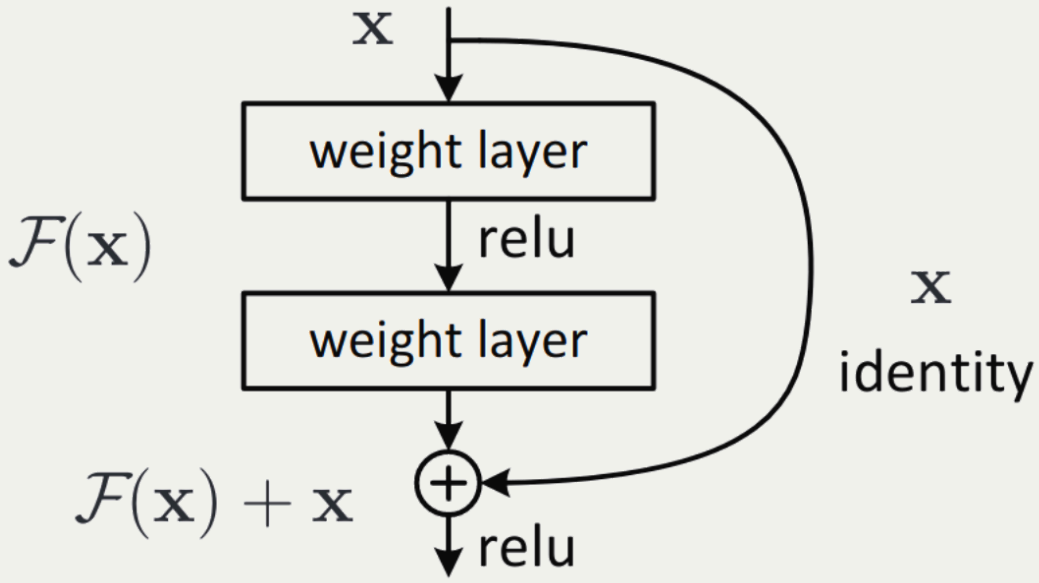
NLPortuguês

Residual Connections



NLPortuguês

Residual Connections



Também conhecidas como *Skip-Connections*.

São utilizadas no modelo Transformer

NLPortuguês

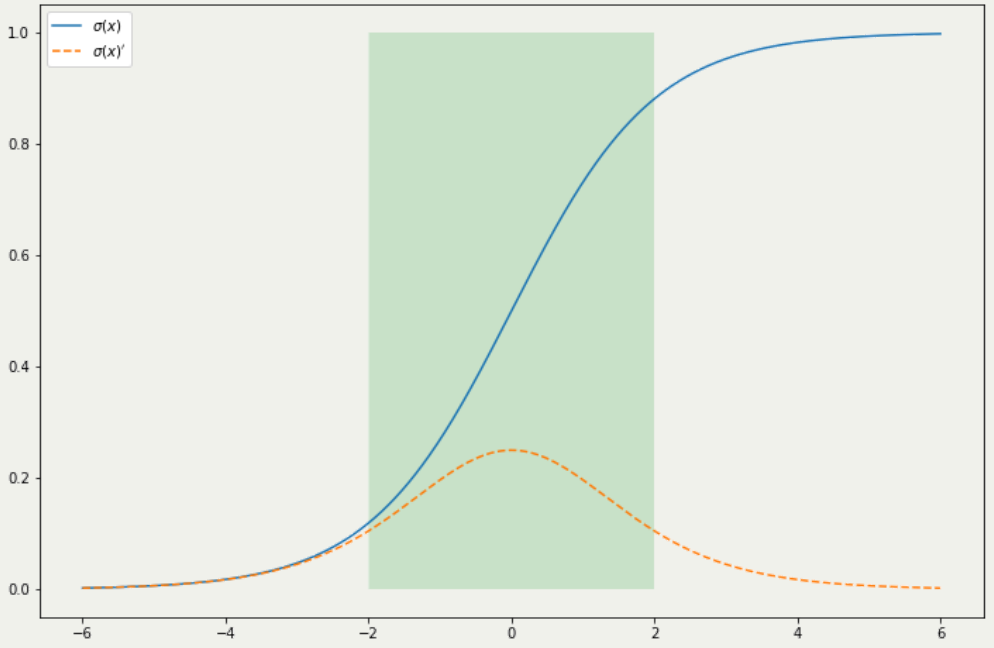
Como combater Vanishing Gradient?

- ReLu
- Conexões Densas
- Conexões Residuais *(Residual Connections)*
- Batch Normalization

NLPortuguês

Batch Normalization

Normalizar ajuda a impedir que as ativações saturem ao confiná-las em uma região.



NLPortuguês

Como combater Vanishing Gradient?

- ReLu
- Conexões Densas
- Conexões Residuais *(Residual Connections)*
- Batch Normalization
- Xavier & He

NLPortuguês

Inicialização dos pesos

Uma inicialização muito baixa nos leva a vanishing gradient e uma inicialização muito grande nos leva a exploding gradient.

Como podemos inicializar os pesos de uma maneira apropriada?

- Inicialização de Xavier Glorot para ativações simétricas (como a tanh)
- Inicialização de Kaiming He para ativações assimétricas (como a ReLu)

NLPortuguês

Como combater Vanishing Gradient?

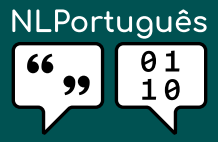
- ReLu
- Conexões Densas
- Conexões Residuais *(Residual Connections)*
- Batch Normalization
- Xavier & He
- LSTM & GRU

NLPortuguês









LSTM & GRU

Parte 1

Marcelo Finger
Alan Barzilay

Roteiro

1. Conceitos de Álgebra Linear
2. LSTMs
3. GRUs
4. Redes Bidirecionais

NLPortuguês

Revisão Rápida de Conceitos de Álgebra Linear

NLPortuguês

Produto Escalar

$$\vec{u} \cdot \vec{v} = \sum_{i=1}^{n} u_i v_i = |\vec{u}||\vec{v}| cos(\theta)$$

Notações alternativas: $\langle \vec{u}, \vec{v} \rangle, \vec{u} \cdot^T \vec{v}, \vec{u}^T \cdot \vec{v}, \vec{u}' \cdot \vec{v}$

Projeção de u sobre v

Operação básica da multiplicação de matrizes: $C = A \times B$

$$c_{ij} = linha_i(A) \cdot coluna_j(B) = A_i \cdot B^j$$

NLPortuguês

Produto de Hadamard

$$\begin{bmatrix} 3 & 5 & 7 \\ 4 & 9 & 8 \end{bmatrix} * \begin{bmatrix} 1 & 6 & 3 \\ 0 & 2 & 9 \end{bmatrix} = \begin{bmatrix} 3\times1 & 5\times6 & 7\times3 \\ 4\times0 & 9\times2 & 8\times9 \end{bmatrix}$$

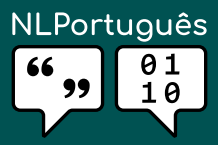
Produto item a item

NLPortuguês









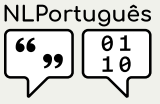
LSTM & GRU

Parte 2

Marcelo Finger
Alan Barzilay

Roteiro

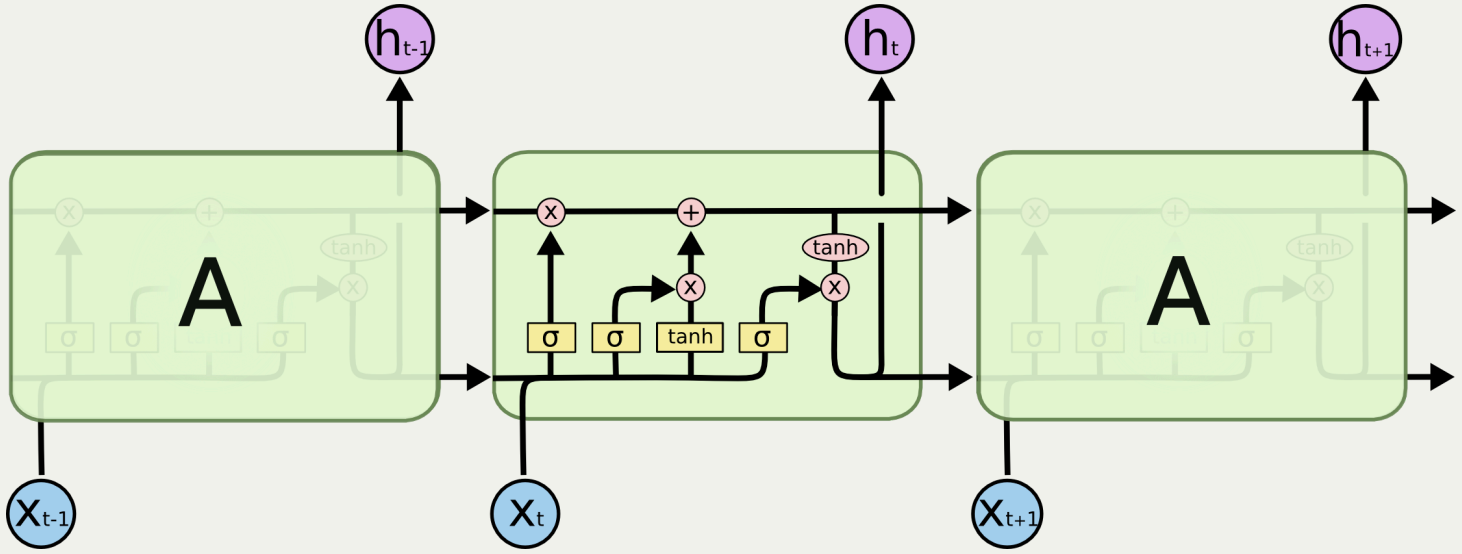
1. Revisão de AL
2. LSTMs
3. GRUs
4. Redes Bidirecionais



LSTM: Long Short Term Memory

NLPortuguês

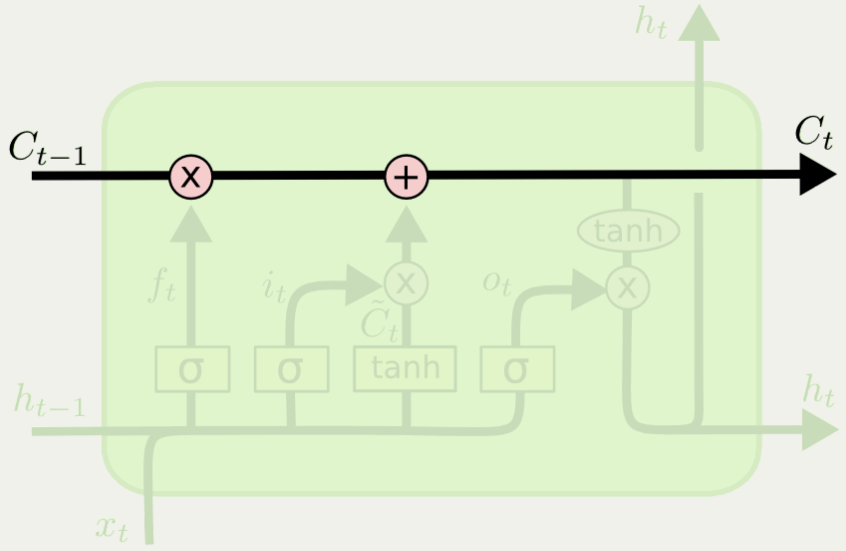
LSTM



NLPortuguês

Long Short Term Memory (LSTM)

A LSTM combate isso ao criar um *cell state* onde informações podem fluir de um estado anterior ao próximo, gerando uma espécie de memória na rede.



NLPortuguês

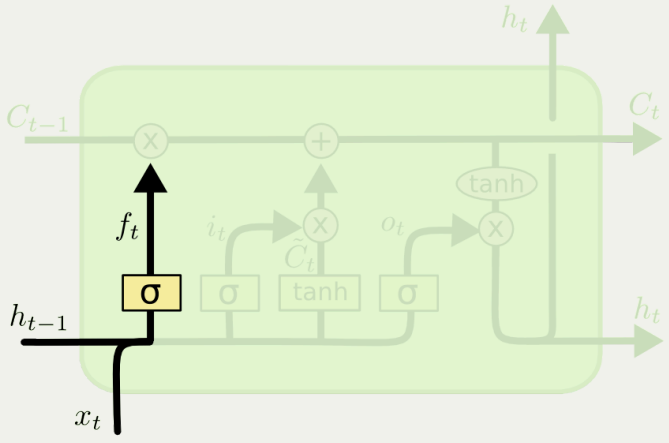
A LSTM possui 3 gates principais:

- Forget gate
- Input gate
- Output gate

NLPortuguês

Forget Gate

O que deve ser esquecido?



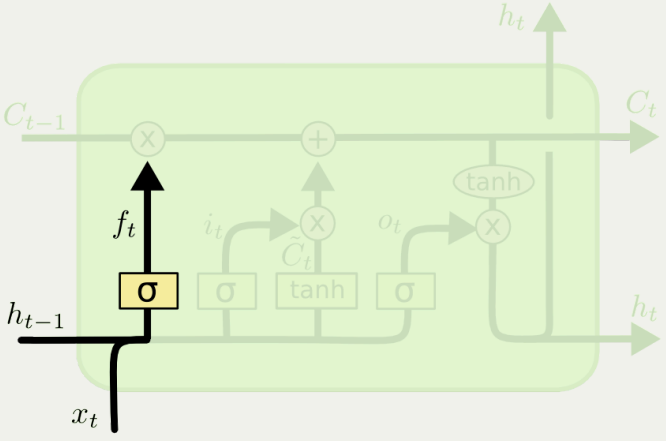
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t]' + b_f\right)$$

NLPortuguês

Forget Gate

O que deve ser esquecido?

Decide o que apagar do *cell state*, valores próximos de 1 serão mantidos e próximos de 0 serão descartados.

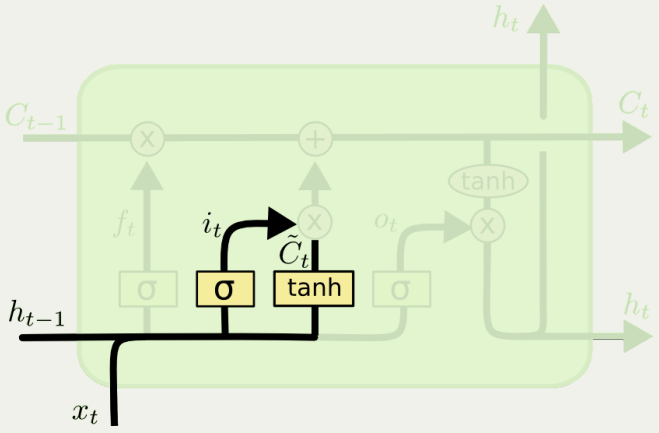


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t]' + b_f\right)$$

NLPortuguês

Input Gate

$\tilde{C}_t$ calcula candidatos a serem armazenados no *cell state*



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t]' + b_i\right)$$

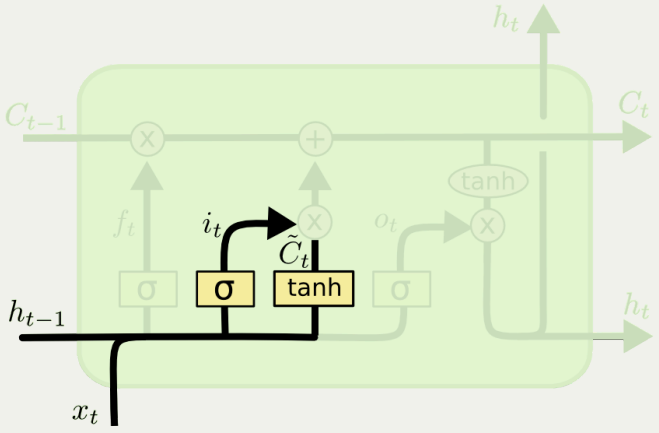
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t]' + b_C)$$

NLPortuguês

Input Gate

$\tilde{C}_t$ calcula candidatos a serem armazenados no *cell state*

O *input gate* decide quais desses valores candidatos devem ser atualizados no *cell state*



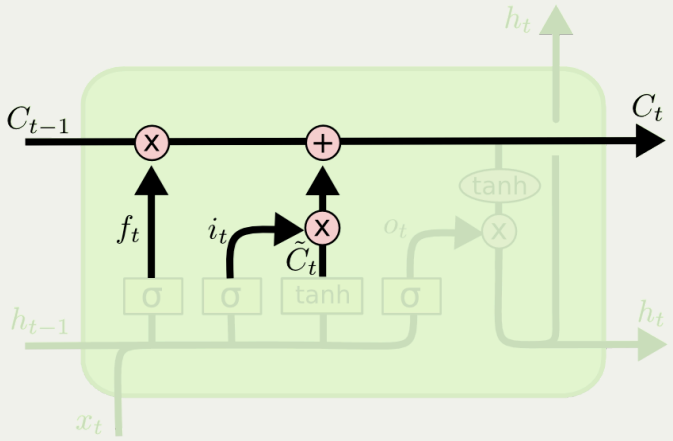
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t]' + b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t]' + b_C)$$

NLPortuguês

Cell State

Os valores “selecionados” do *cell state* anterior e do candidato serão combinados para formar o novo *cell state*

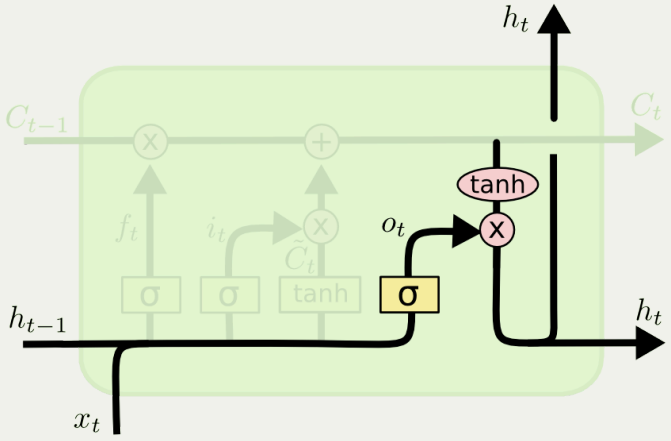


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

NLPortuguês

Output Gate

Novamente uma sigmoide, “seleciona” os elementos do *cell state* que serão utilizados como output.



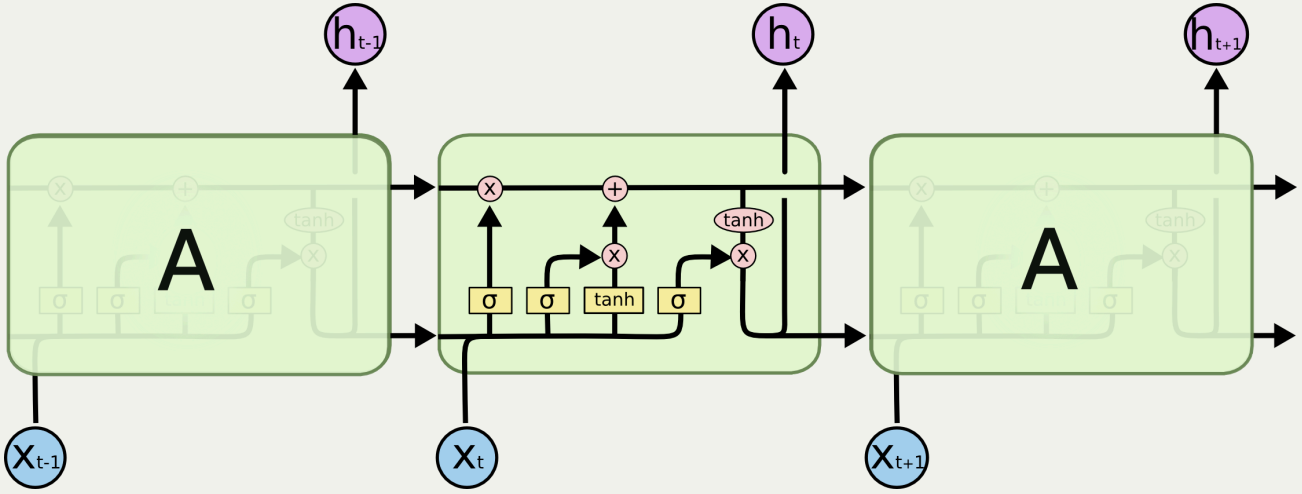
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right]' + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

NLPortuguês

Recapitulando LSTM

O *forget gate* decide o que manter das etapas anteriores, o *input gate* decide que informação manter da etapa atual e o *output gate* determina o próximo *hidden state*.

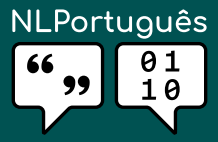


NLPortuguês









LSTM & GRU

Parte 3

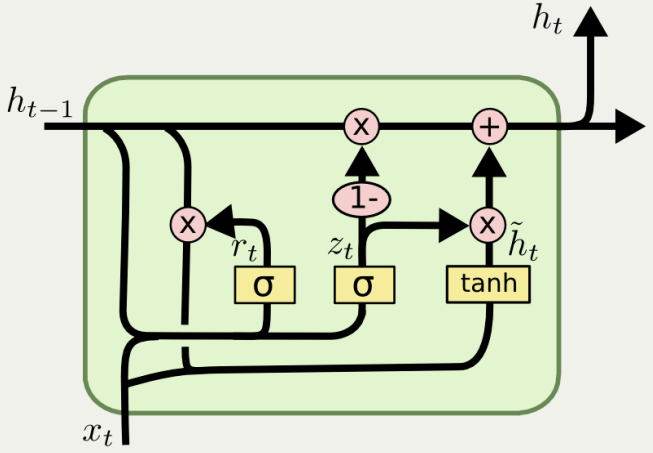
Marcelo Finger
Alan Barzilay

Roteiro

1. Revisão de AL
2. LSTMs
3. GRUs
4. Redes Bidirecionais

GRU: Gated Recurrent Unit

GRU - Gated Recurrent Unit



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

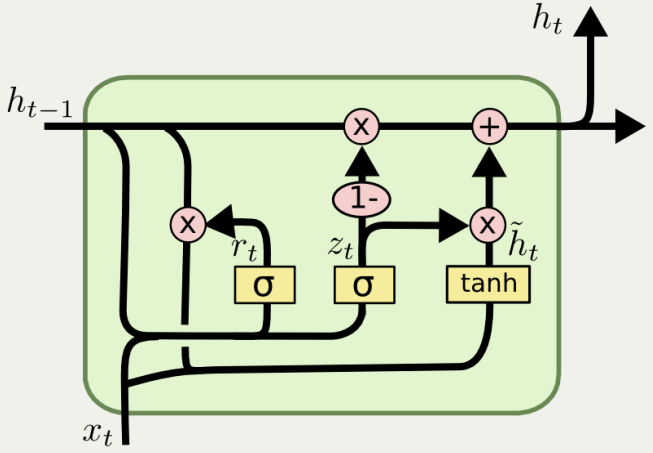
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU - Gated Recurrent Unit

Forget e input gates são unidos em um unico *update gate z*



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

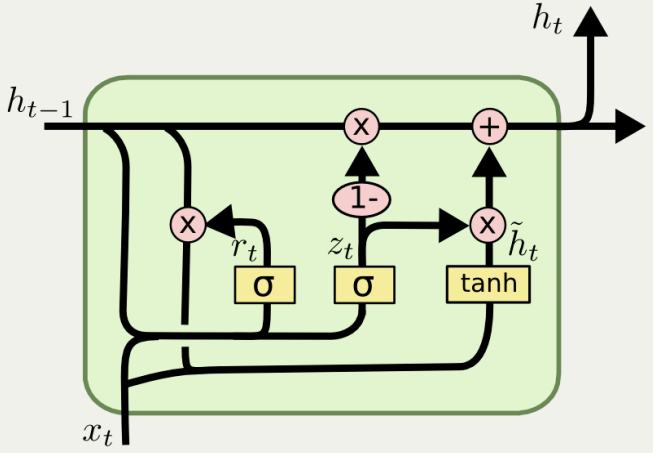
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU - Gated Recurrent Unit

Forget e input gates são unidos em um unico *update gate z*

Cell state e *hidden state* passam a ser uma única entidade



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Usos de LSTMs e GRUs

- São tipicamente usadas em modelos seq2seq
  - Uma LSTM/GRU como encoder, uma LSTM/GRU como decoder
- Quando treinadas com grandes quantidades de dados, geram Modelos (Neurais) Recorrentes de Linguagem
- O código gerado pode ser ligado a outras redes para classificação, regressão ou tradução

GRU *vs* LSTM

Quando usar cada uma?

LSTM é um pouco mais poderosa (captura mais padrões do contexto)

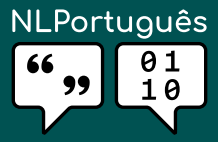
GRU é mais rápida de ser treinada

Dificilmente você encontrará uma RNN pura sendo usada, mesmo para arquiteturas encoder decoder é mais comum usarmos LSTM, GRU, ou outras redes derivadas.









LSTM & GRU

Parte 4

Marcelo Finger
Alan Barzilay

Roteiro

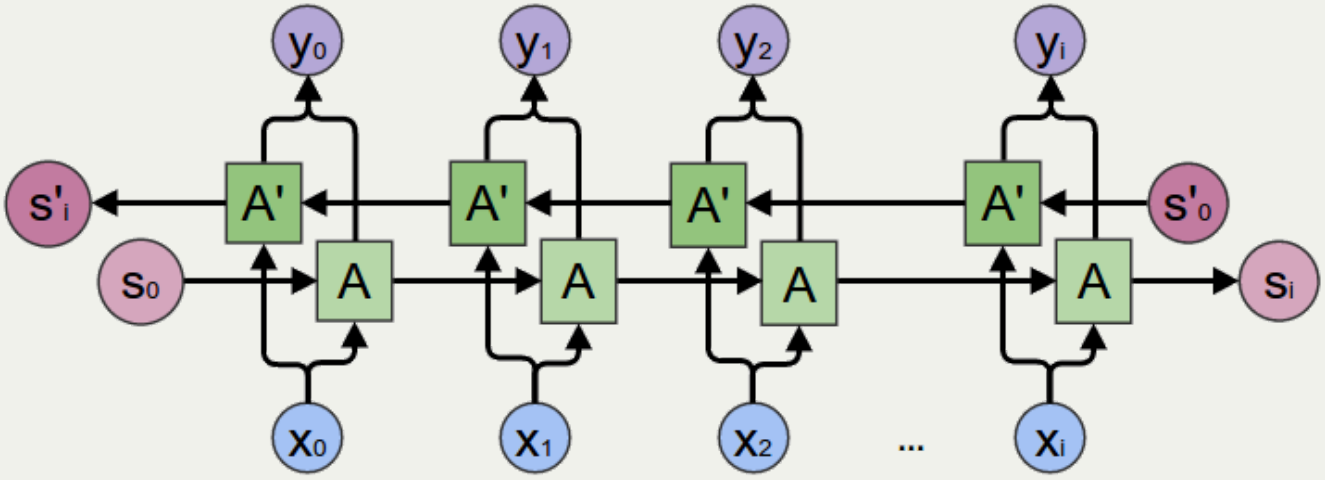
1. Revisão de AL
2. LSTMs
3. GRUs
4. Redes Bidirecionais

Redes Neurais Bidirecionais

Redes Bi-direcionais

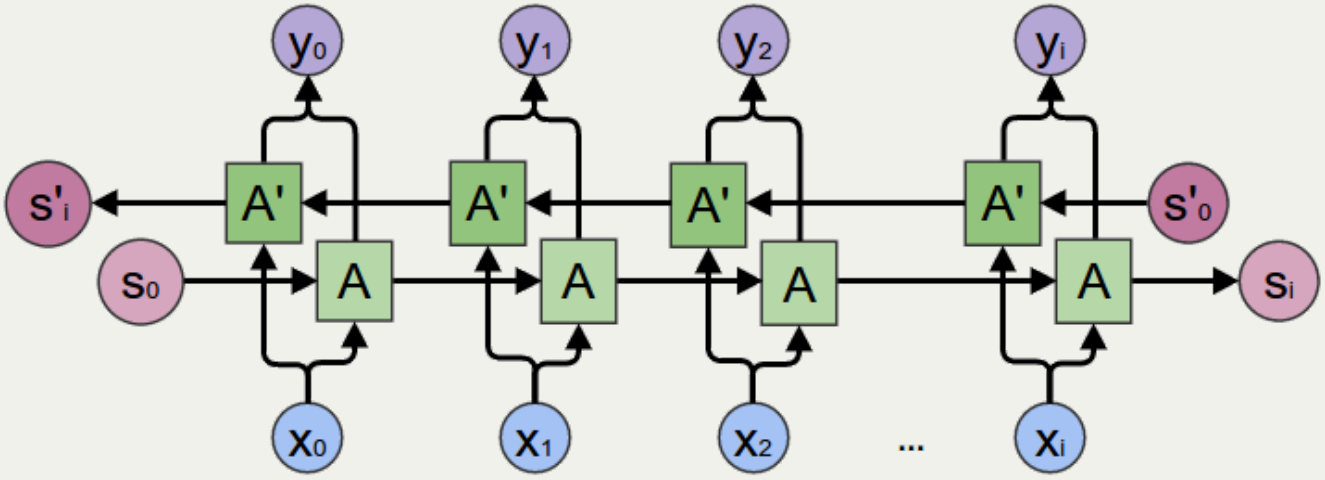
Mesma ideia, mas as 2 redes recebem as tokens em ordem contrária e seus *hidden states* no final são concatenados.

Isso permite incluir contexto de ambas as direções.



Redes Bi-direcionais

Adicionar probabilidades condicionais em ambas as direcoes aqui



Aplicações

- Redes Bidirecionais podem ser usadas como encoders, da mesma forma que as redes unidirecionais
- Modelos de Linguagem mais sensíveis ao contexto
- Podem ter várias camadas
- Maior complexidade do modelo requer mais dados
- Podem ser usadas com Atenção Neural também (próximo curso)